SDD-Crash-Course: Complete End-to-End Engineering Course

Author: Alejandro Kaspar - AI Forward Deployed Engineer | Conductor, OpenSpec & SPEC.md on TaskFlow

1. Conductor (Context & TDD Engine)

Unit of Work: Track (`conductor/tracks/<id>/`)

- Persistent repo constitution (`product.md`, `tech-stack.md`, `workflow.md`).
- Interactive 7-Section Track `spec.md` & phased `plan.md`.
- Strict **Red** → **Green** → **Refactor** TDD execution (`/conductor:implement`) with `git notes` & SHA check-points.

2. OpenSpec (Delta Contract Engine)

Unit of Work: Capability Delta (`openspec/changes/<id>/`)

- Two-Plane isolation: `openspec/specs/` (Current Truth) vs. `openspec/changes/` (Proposed Delta).
- Normative RFC 2119 (`SHALL`, `MUST`) + 4-hashtag `#### Scenario:` (`GIVEN/WHEN/THEN`).
- Interactive Socratic Grilling + `/opsx:verify` & `/opsx:archive`.

3. SPEC.md (Single Source of Truth)

Unit of Work: Whole-System Blueprint (`SPEC.md`)

- Canonical 8-Section architecture, schema, API, and deployment specification.
- Separates **Essential Domain Intent** from incidental implementation details.
- Verified by the **Clean-Room Rebuild Test** against read-only adversarial tests.

7-Module Step-by-Step Curriculum: Building `TaskFlow` Across 6 Progressive Python Labs

- **Module 01:** Foundations, Spec Triangle, Socratic Grilling & Dual-Engine Wiring
- **Module 02 (Lab 01):** Greenfield `TodoList` Specs & ADRs
- **Module 03 (Lab 02):** Brownfield `TaskOrganizer` Tag Normalization
- **Module 04 (Lab 03):** `HabitTracker` Streak TDD
- **Module 05 (Lab 04):** `DailyPlanner` Off-by-One Drift & Scorecard Refactoring
- **Module 06 (Lab 05):** `NotesBoard` SDD Code Review & Stacked PRs
- **Module 07 (Lab 06):** `TaskFlowBoard` `SPEC.md` Clean-Room Rebuild

Specificational Coding vs. Conversational "Vibe Coding"

Making source code a compiled artifact of a living, version-controlled specification

Anti-Pattern: Conversational Coding ("Vibe Coding")

1. **Prompts Disappear After Merge:** Chat history vanishes when the session closes, leaving future engineers with code and zero recorded intent.
2. **Silent Edge-Case Guessing:** Agent guesses whether blank task titles are allowed or whether a 100% full daily schedule should be rejected.
3. **Sycophantic Self-Testing:** Unconstrained prompts write weak assertions that rubber-stamp broken implementation code.
4. **Collateral Regressions:** Agent reverses note ordering while adding a feature.

Solution: Specificational Coding (SDD + SDT)

1. **Persistent Spec Artifact:** The user edits and approves `spec.md`; the agent compiles that spec into code (**SDD**) and 1:1 tests (**SDT**).
2. **Socratic Grilling Before Code:** Agent asks 3–5 targeted boundary questions before generating any specification or code.
3. **Surgical Scope & Stack Locks:** Lock `Out of Scope` + `tech-stack.md`; verify at small Phase Checkpoints & Stacked PR batches.
4. **Immutable Verifiers (`chmod -w`):** 1:1 `test_reqXXXX_*` + read-only tests.

Karpathy's 4 Coding Agent Rules (Enforced in `conductor/workflow.md`)

• **(1) Think Before Coding:** Surface ambiguities via Socratic Q&A instead of guessing · **(2) Simplicity First:** Write 50 lines of clean stdlib Python instead of 500 lines of bloat · **(3) Surgical Changes:** Honor `Out of Scope` strictly · **(4) Goal-Driven Execution:** Loop against `test_reqXXXX_*` and `chmod -w` verifiers.

The Spec Triangle (`Apply`, `Sync`, `Clean`) & 3 SDD Maturity Levels

How Specifications, Code, and Tests stay synchronized across the full software lifecycle

Action 1: Apply (*Act on Spec*)

Spec → Code + Tests

- Triggered by `/conductor:implement` and `/opsx:apply`.
- Compiles approved `REQ-XXXX` contracts into failing `test_reqXXXX_*` tests first (**Red**), then minimal production code (**Green**).
- Practiced in **Lab 01** and **Lab 03**.

Action 2: Sync (*Sync Spec*)

Code ↔ Spec Reconciliation

- Triggered by `/opsx:explore`, `/opsx:verify`, and `/conductor:review`.
- Extracts baseline specs from brownfield code (**Lab 02**) or detects spec-to-code drift (**Lab 04** & **Lab 05**).
- Prevents spec rot across sprints.

Action 3: Clean (*Archive & Rebuild*)

Delta Merge & Clean-Room SSOT

- Triggered by `/opsx:archive` and the Clean-Room Rebuild Test.
- Folds active delta changes into `openspec/specs/` (**Lab 05**) and proves `SPEC.md` can regenerate the app from scratch (**Lab 06**).

The 3 Levels of SDD Maturity Across Your Repository

- **Level 1 (Spec-First):** Write prompt/spec once, discard after coding (avoid — rots immediately).
- **Level 2 (Spec-Anchored):** Bidirectional delta sync via **OpenSpec** (`openspec/specs/`) + **Conductor** (`conductor/tracks/`) — default for daily engineering.
- **Level 3 (Spec-as-Source / SSOT):** Canonical 8-Section **`SPEC.md`** verified by deleting `src/` and passing read-only `chmod -w` adversarial tests in `/tmp/`.

Conductor: Context-Driven Development & TDD Track Execution

"Measure Twice, Code Once" — Persistent repository context, Plan Mode sandboxing, and Git-aware execution

Step 1: `/conductor:setup`

One-Time Repo Constitution

- Scans greenfield or brownfield repo (respecting `.gitignore`).
- Generates `conductor/index.md`, `product.md`, `product-guidelines.md`, `tech-stack.md`, `workflow.md`, and `tracks.md`.

Step 2: `/conductor:newTrack`

Sandboxed Spec & Phased Plan

- Enforces `policies/conductor.toml` (blocks writes outside `conductor/`).
- Conducts 3–5 interactive Q&A turns.
- Emits 7-section `spec.md`, hierarchical `plan.md`, and `metadata.json`.

Step 3: `/conductor:implement`

Strict Red-Green-Refactor TDD

- Marks task `[~]`, writes failing `test_reqXXXX_*` first (**Red**).
- Writes minimal code (**Green**), checks `>80%` coverage (`CI=true`).
- Commits code, attaches `git notes`, records `[x] <7-char-sha>`.

Day-2 Engineering Commands: Auditing, Principal-Engineer Review & Logical Git Revert

- `/conductor:status` — Audits `tracks.md` and `plan.md` state markers (`[ ]`, `[~]`, `[x] sha`, `[checkpoint: sha]`) against Git history.
- `/conductor:review` — Audits diff against `spec.md`, `tech-stack.md`, and `code_styleguides/`; appends auto-fixes to `## Phase: Review Fixes`.
- `/conductor:revert` — 4-Phase Git-aware logical rollback of an entire Track, Phase, or Task (reconciling implementation + plan commits).

OpenSpec: Two-Plane Architecture, Socratic Grilling & Delta Algebra

Separating “Current System Truth” (`openspec/specs/`) from “Proposed Changes” (`openspec/changes/`)

Plane 1: `openspec/specs/<capability>/spec.md`

The Living System of Record ("The Elephant")

- Organized by capability domain (`todo-list`, `task-organizer`, `notes-board`), never by sprint ticket.
- Every requirement uses `### Requirement: [REQ-XXXX] <Title>` + uppercase **RFC 2119** (`SHALL`, `MUST`, `MUST NOT`).
- Every testable behavior uses **4-hashtag** `#### Scenario:` with `GIVEN`, `WHEN`, `THEN`.

Plane 2: `openspec/changes/<change-id>/`

Socratic Grilling & Isolated Delta Bundle ("The Goldfish")

- **1. Semantic Diffing & Constraint Extraction:** Compares baseline `openspec/specs/` against the requested change.
- **2. Interactive Socratic Prompting ("The Grill"):** Asks 3–5 boundary questions (`Q1..Q3`) before generating files.
- **3. Delta Bundle:** Emits `proposal.md`, `design.md`, `specs/<cap>/spec.md` (`ADDED/MODIFIED/REMOVED/RENAMED`), and `tasks.md`.

The 99% / 1% CLI Rule & Deterministic 5-Stage Archive Merge (`specs-apply.js`)

- **1% Terminal CLI:** Run `openspec init` once and `openspec validate --all --strict` in CI. **99% Slash Skills:** `/opsx:explore` → `/opsx:propose` → `/opsx:apply` → `/opsx:verify` → `/opsx:archive`.
- **Archive Execution Order:** **(1)** Pre-Validate Cross-Section Conflicts → **(2)** `RENAMED` → **(3)** `REMOVED` → **(4)** `MODIFIED` (full replacement!) → **(5)** `ADDED`.

Wiring Conductor + OpenSpec + `@writing-arch-specs`

How `conductor/workflow.md` invokes dedicated spec skills and aligns tracks with `openspec/specs/`

1. `conductor/workflow.md` & `index.md` Handshake

```
# conductor/index.md
- [Writing Arch Specs Skill](../.agents/skills/writing-arch-specs/SKILL.md)
- [OpenSpec Canonical Specs](../openspec/specs/)

# conductor/workflow.md
1. Skill Activation: Read .agents/skills/writing-arch-specs/SKILL.md
2. Contract Alignment: Map track spec.md 1:1 to REQ-XXXX in openspec/specs/
3. TDD Loop: Write failing test_reqXXXX_* first (Red -> Green -> Refactor)

4. Guardrails: Think Before Coding, Simplicity First, Surgical Changes, Immutable Verifiers
```

2. The 7-Section Track `spec.md` Anatomy

1. **Overview:** Problem statement, business goal, target service.
2. **Architecture & Component Topology:** Data structures & boundaries.
3. **Functional Requirements:** Traceable `REQ-XXXX` contracts.
4. **Non-Functional Requirements:** Complexity, stdlib-only constraints.
5. **Acceptance Criteria:** Binary pass/fail scenarios.
6. **Out of Scope:** Explicit deferred items (prevents scope creep!).
7. **Verification Commands:** Exact non-interactive `CI=true` commands.

Why Dual-Engine Beats Either Tool Alone

- **OpenSpec alone** gives you clean capability contracts (`openspec/specs/`) and delta merges, but lacks Conductor's repo-wide `tech-stack.md` lock, `git notes` audit trail, and 4-phase logical `/conductor:revert`.
- **Conductor alone** gives you disciplined TDD track execution, but without OpenSpec its track specs remain isolated per ticket instead of merging into a permanent capability tree. **Together, they form a closed loop.**

Lab 01 Step 1: Auditing the Vague Proposal & Socratic Grilling

Walking through `labs/lab_01_greenfield_proposals_to_specs/proposal.md` (**TaskFlow Core To-Do List**)

Input: The Vague Product Note (`proposal.md`)

```
"We are launching TaskFlow, a lightweight productivity app.
For the MVP, build a Python TodoList class where users can
add tasks, mark tasks as completed, list pending or completed
tasks, and delete tasks when they are no longer needed."
```

Why Vibe Coding Fails Here: Without a spec, an agent guesses whether `" "` is a valid task title, whether `complete_task(999)` raises an error or returns `False`, and whether deleted IDs get reused!

Step 1 Action: Socratic Grilling ("The Grill")

In Plan Mode, the agent pauses and asks **3 Clarifying Questions**:

Q1 (Blank & Whitespace Titles): Should `add_task(" Buy milk ")` trim whitespace, and what if the trimmed title is empty? → *Trim whitespace; raise `ValueError` if empty.*

Q2 (Missing Task IDs): How should `complete_task(id)` vs. `delete_task(id)` handle unknown IDs? → *`complete_task` raises `KeyError`; `delete_task` returns `bool` (`True`/`False`).*

Q3 (Ordering): How are filtered lists ordered? → *Creation order (`id = 1, 2, ...`).*

Hands-On Terminal & Prompt Recipe for Lab 01 Step 1

- **Inspect Starting Proposal:** `cd labs/lab_01_greenfield_proposals_to_specs && cat proposal.md`
- **Slash Command:** Run `/opsx:explore` to resolve the 3 boundary questions, then run `/opsx:propose "add-todo-list-mvp"` and `/conductor:newTrack "Implement TaskFlow Core TodoList MVP"`.

Lab 01 Step 2: Authoring the OpenSpec Capability Contract

Writing `expected_output/openspec/specs/todo-list/spec.md` with `REQ-0001..REQ-0004`

Exact OpenSpec Contract Excerpt (REQ-0001 & REQ-0002)

```
### Requirement: [REQ-0001] Task Creation & Title Validation
The TodoList.add_task(title) method SHALL strip leading/trailing
whitespace, assign a sequential integer `id` starting at `1`,
and MUST raise `ValueError` if the trimmed title is empty.

#### Scenario: Creating valid tasks and rejecting blank titles
- **GIVEN** a new `TodoList` instance
- **WHEN** `add_task(" Write OpenSpec ")` is called
- **THEN** it SHALL return `Task(id=1, title="Write OpenSpec", completed=False)`
  and calling `add_task(" ")` SHALL raise `ValueError`.

### Requirement: [REQ-0002] Marking a Task Completed
`complete_task(task_id)` SHALL set `completed=True` on the matching

task and MUST raise `KeyError` when `task_id` does not exist.
```

Complete Requirement Matrix (REQ-0001..REQ-0004)

- **REQ-0001 (Task Creation & Validation):** Trims `title`, assigns sequential `id` ($1, 2, \ldots$), sets `completed=False`, raises `ValueError` on blank string.
- **REQ-0002 (Task Completion):** `complete_task(task_id)` sets `completed=True` and returns the updated `Task`, or raises `KeyError` if missing.
- **REQ-0003 (Pending & Completed Filters):** `get_pending()` and `get_completed()` return tasks in creation order.
- **REQ-0004 (Task Deletion):** `delete_task(task_id)` removes the task and returns `True`, or returns `False` if not found.

Structural Validation Gate Check in self_diagnose.sh

- Verify every requirement header matches `### Requirement: [REQ-000X]`, contains uppercase `SHALL`/`MUST`, and includes a 4-hashtag `#### Scenario:` block with `GIVEN`, `WHEN`, and `THEN`.

Lab 01 Step 3: Recording Architecture Decision Records (ADRs)

Evaluating storage and ID-assignment trade-offs in `ADR-0001-storage-engine.md` and `ADR-0002-id-assignment.md`

`ADR-0001`: In-Memory Dictionary Storage Engine

Context: Need $O(1)$ task lookup by `task_id` while preserving chronological insertion order for `get_pending()` and `get_completed()`.

Options Evaluated:

1. **Insertion-Ordered `dict[int, Task]` (ACCEPTED):** Python 3.7+ `dict` guarantees insertion order while providing $O(1)$ lookup and deletion by `task_id`.
2. **Plain `list[Task]` (REJECTED):** Requires $O(n)$ linear scan on every `complete_task` and `delete_task` call.
3. **SQLite Database (DEFERRED):** Unnecessary I/O setup for in-memory MVP.

`ADR-0002`: Monotonic Sequential ID Assignment

Context: When task `1` is deleted via `delete_task(1)`, what ID should the next created task receive?

Options Evaluated:

1. **Monotonic Counter `_next_id += 1` (ACCEPTED):** Never reuses deleted IDs; deterministic $(1, 2, 3 \ldots)$ in unit tests and prevents stale-ID collisions.
2. **`len(self._tasks) + 1` (REJECTED):** Collides with existing IDs as soon as a middle task is deleted!
3. **UUIDv4 Strings (REJECTED):** Verbose and non-sequential for CLI assertions.

Why ADRs Are Mandatory Even for Simple Services

- Specifications (`spec.md`) capture **WHAT** the system must do; ADRs capture **WHY** naive shortcuts (like `len(tasks) + 1`) were rejected—preventing future AI sessions from introducing ID-collision bugs!

Lab 01 Step 4: Authoring the 7-Section Conductor Track `spec.md` & `plan.md`

Bridging OpenSpec contracts into Conductor's phased Red-Green-Refactor execution plan

7-Section Conductor Track `spec.md` Structure

```
## 1. Overview
Implement TaskFlow Core TodoList satisfying REQ-0001..REQ-0004.
## 2. Architecture & Component Topology
Task dataclass + TodoList backed by insertion-ordered dict[int, Task].
## 3. Functional Requirements
Maps 1:1 to REQ-0001..REQ-0004 in openspec/specs/todo-list/spec.md.
## 4. Non-Functional Requirements
O(1) lookup by ID; Python 3.10+ stdlib only.
## 5. Acceptance Criteria
All 4 unit tests pass with 100% requirement traceability.
## 6. Out of Scope
SQLite persistence; multi-user authentication; recurring tasks.
## 7. Verification Commands

CI=true python3 -m unittest discover -s expected_output -p "test_*.py" -v
```

Phased Conductor `plan.md` with Checkpoints

```
## Phase 1: Task Creation & Completion [checkpoint: a1b2c3d]
- [x] Task 1.1: Implement Task dataclass & add_task() (REQ-0001) 11a2b3c
- [x] Task 1.2: Implement complete_task() & KeyError (REQ-0002) 22b3c4d
- [x] Task: Conductor - User Manual Verification 'Phase 1'

## Phase 2: Filtering & Deletion [checkpoint: d4e5f6a]
- [x] Task 2.1: Implement get_pending() & get_completed() (REQ-0003) 33c4d5e
- [x] Task 2.2: Implement delete_task() boolean return (REQ-0004) 44d5e6f

- [x] Task: Conductor - User Manual Verification 'Phase 2'
```

Human-in-the-Loop Gate Before Code Generation

- Notice Section 6 (`Out of Scope`): explicitly excluding SQLite persistence and user auth keeps the agent focused on the 50-line domain model. Once you approve `spec.md` and `plan.md`, you trigger `/conductor:implement`.

Lab 01 Step 5: Implementing `todo_list.py` & Running `./self_diagnose.sh`

Inspecting the clean 50-line reference implementation and verifying all 4 traceable unit tests

Reference Implementation (`expected_output/todo_list.py`)

```
class TodoList:
  def __init__(self) -> None:
    self._tasks: dict[int, Task] = {}
    self._next_id: int = 1

  def add_task(self, title: str) -> Task:
    cleaned = title.strip() if isinstance(title, str) else ""
    if not cleaned: raise ValueError("Task title must not be empty")
    task = Task(id=self._next_id, title=cleaned, completed=False)
    self._tasks[self._next_id] = task
    self._next_id += 1
    return task

  def complete_task(self, task_id: int) -> Task:
    if task_id not in self._tasks: raise KeyError(f"Unknown task: {task_id}")
    old = self._tasks[task_id]
    updated = Task(id=old.id, title=old.title, completed=True)
    self._tasks[task_id] = updated

    return updated
```

Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_01_greenfield_proposals_to_specs/self_diagnose.sh
=== Lab 01 Self-Diagnosis: Greenfield Proposals to Specs ===
test_req0001_add_task_trims_title_and_rejects_blank ... ok
test_req0002_complete_task_marks_done_and_raises_on_missing ... ok
test_req0003_get_pending_and_completed_preserve_creation_order ... ok
test_req0004_delete_task_removes_entry_and_returns_boolean ... ok
----------------------------------------------------
Ran 4 tests in 0.001s -- OK
[PASS] Lab 01 verified: OpenSpec contract, 2 ADRs, Conductor track,

       and 4/4 TodoList unit tests passed!
```

Lab 01 Key Takeaway

- By spending 3 minutes on Socratic Grilling (`/opsx:propose`), 2 ADRs, and a 7-section Conductor `spec.md`, the 50-line `TodoList` implementation and 4 unit tests pass on the very first run.

Lab 02 Step 1: Auditing an Undocumented Brownfield Codebase

Inspecting `labs/lab_02_brownfield_spec_recovery/service/task_organizer.py` via `/opsx:explore`

The Brownfield Challenge: Code Exists, Specs Don't

- You inherit `service/task_organizer.py` (`TaskFlow`'s Tag & Priority Organizer) and `service/test_task_organizer.py`, with **zero specification files**.
- **Why Direct Agent Edits Are Dangerous:** If you ask an agent to "simplify tag handling" without a baseline spec, it might replace the normalization loop with `tuple(tags)`—silently allowing duplicate `"Urgent"` and `"urgent"` tags and breaking case-insensitive search!

Inspecting `service/task_organizer.py` in Explore Mode

```
PRIORITY_RANK = {"high": 0, "medium": 1, "low": 2}

class TaskOrganizer:
  def add_task(self, title: str, priority: str = "medium",
               tags: Iterable[str] = ()) -> OrganizedTask:
    norm_priority = priority.strip().lower()
    if norm_priority not in PRIORITY_RANK: raise ValueError(...)
    normalized_tags: list[str] = []
    for raw in tags:
      tag = raw.strip().lower()
      if tag and tag not in normalized_tags:

        normalized_tags.append(tag)
```

Step 1 Command & Read-Only Investigation Rule (*Sync Spec*)

- Run `/opsx:explore` on `labs/lab_02_brownfield_spec_recovery/service/`. Remember: `/opsx:explore` is strictly read-only—it maps every validation rule, normalization invariant, and sort tie-breaker before writing any spec.

Lab 02 Step 2: Extracting the 4 Hidden Behavioral Invariants

Mapping every method and guard in `task_organizer.py` to `REQ-0001` through `REQ-0004`

Invariants `REQ-0001` & `REQ-0002` (Priority & Tags)

- **`REQ-0001` (Title & Priority Validation):** `add_task(title, priority, tags)` trims `title`, normalizes `priority` to lowercase `{"high", "medium", "low"}`, and raises `ValueError` on blank titles or unsupported priority levels.
- **`REQ-0002` (Tag Normalization & Deduplication):** All tags MUST be stripped, lowercased, filtered for non-empty strings, and deduplicated in first-seen order (e.g. `["Work", " work ", "URGENT"]` → `("work", "urgent")`).

Invariants `REQ-0003` & `REQ-0004` (Filtering & Sorting)

- **`REQ-0003` (Case-Insensitive Tag Filtering):** `filter_by_tag(tag)` normalizes the query tag (`strip().lower()`) and returns matching tasks in creation order.
- **`REQ-0004` (Priority-Ranked Ordering):** `list_by_priority()` orders tasks by priority rank (`high` → `medium` → `low`) and breaks ties by ascending creation `id`.

Brownfield Extraction Rule: Capture Essential Behavior, Not Incidental Details

- Notice that `REQ-0004` specifies the *observable sort order* (`high` before `medium` before `low`, tie-broken by `id`) rather than hardcoding internal dictionary variable names in the contract.

Lab 02 Step 3: Why Un-Normalized Tags Break Filtering (ADR-0001)

Documenting the data-hygiene rationale in `expected_output/ADR-0001-tag-normalization.md`

The Case-Mismatch & Duplicate Tag Hazard

1. User A tags a task with `["Urgent", " backend "]`.
2. User B tags a task with `["urgent", "URGENT"]`.
3. Without ingestion normalization, User B's task stores duplicate tags, and calling `filter_by_tag("urgent")` **misses User A's `"Urgent"` task** and fails to match `" backend "` due to trailing spaces!

Write-Time Tag Normalization Solution (ADR-0001)

1. Normalize every tag at write time via `raw.strip().lower()`.
2. Discard empty/whitespace-only tags (`""`).
3. Deduplicate while preserving first-seen order into an immutable `tuple[str, ...]`.
4. Normalize query strings in `filter_by_tag(query)` with the exact same `query.strip().lower()` rule so lookups are 100% deterministic.

Why Recovering `ADR-0001-tag-normalization.md` Protects Your Codebase

- Recording this write-time normalization decision in `ADR-0001-tag-normalization.md` ensures no future contributor or AI agent replaces the normalization loop with a raw `set(tags)` (which loses tag order and keeps case duplicates).

Lab 02 Step 4: Authoring `openspec/specs/task-organizer/spec.md` & `design.md`

Writing the normative 4-hashtag Gherkin scenarios and component design doc

Recovered Spec (`openspec/specs/task-organizer/spec.md`)

```
### Requirement: [REQ-0002] Tag Normalization and Deduplication
The TaskOrganizer SHALL normalize every tag by stripping whitespace
and converting to lowercase, MUST ignore empty tags, and SHALL
deduplicate tags while preserving first-seen order.

#### Scenario: Mixed-case and duplicate tags are normalized on creation
- **GIVEN** a `TaskOrganizer` instance
- **WHEN** `add_task("Fix login", tags=["Urgent", " urgent ", "Backend", ""])`
  is called

- **THEN** the returned task SHALL have `tags == ("urgent", "backend")`.
```

Recovered Design Document (`expected_output/design.md`)

- **Context:** Documents `TaskOrganizer` and the immutable `OrganizedTask(id, title, priority, tags)` dataclass.
- **Goals / Non-Goals:** Guarantees deterministic tag normalization and two-key priority sorting `(PRIORITY_RANK[t.priority], t.id)`; full-text regex search is a Non-Goal.
- **Immutability Invariant:** Storing `tags` as a `tuple[str, ...]` prevents callers from mutating task tags after creation.

Baseline Spec Promotion

- Once verified, your recovered `spec.md` lives permanently at `openspec/specs/task-organizer/spec.md`. The brownfield subsystem is now upgraded to **SDD Maturity Level 2 (Spec-Anchored)**!

Lab 02 Step 5: Verifying Zero-Regression Brownfield Recovery

Running `./labs/lab_02_brownfield_spec_recovery/self_diagnose.sh`

What `lab_02/self_diagnose.sh` Checks

1. **Normative Spec Completeness:** Confirms `REQ-0001` through `REQ-0004` are present in `expected_output/openspec/specs/task-organizer/spec.md`.
2. **RFC 2119 & 4-Hashtag Gherkin Compliance:** Confirms uppercase `SHALL`/`MUST` and `#### Scenario:` blocks with `GIVEN`, `WHEN`, `THEN`.
3. **ADR & Design Completeness:** Confirms `ADR-0001-tag-normalization.md` and `design.md` document write-time tag normalization.
4. **Brownfield Test Suite Execution:** Runs all 4 unit tests in `service/test_task_organizer.py`.

Terminal Output (`./self_diagnose.sh`)

```
$ ./labs/lab_02_brownfield_spec_recovery/self_diagnose.sh
=== Lab 02 Self-Diagnosis: Brownfield Spec Recovery ===
test_req0001_validates_title_and_priority ... ok
test_req0002_normalizes_and_deduplicates_tags ... ok
test_req0003_filters_by_tag_case_insensitively ... ok
test_req0004_lists_tasks_by_priority_then_creation_id ... ok
----------------------------------------------------------
Ran 4 tests in 0.001s -- OK
[PASS] Lab 02 verified: Brownfield unit tests and recovered OpenSpec

       REQ-0001..REQ-0004 + ADR-0001 verified!
```

Lab 02 Key Takeaway

• Never unleash a coding agent on legacy code blind. Spend 5 minutes running `/opsx:explore` (*Sync Spec*) to recover `spec.md` and `ADR-0001` first—then every future change is anchored to verified invariants.

Lab 03 Step 1: Inspecting the Approved Habit Tracker Contract

Walking through `labs/lab_03_from_specs_to_tdd_code/specs/spec.md` (**TaskFlow HabitTracker**)

Daily Streak Calculation Rules (`specs/spec.md`)

- **Same-Day Check-In (Idempotent):** Checking in twice on the same integer `day` (e.g. `day=10` then `day=10`) keeps the streak unchanged.
- **Consecutive Next-Day Check-In (`day == last_day + 1`):** Increments the current streak by `+1` (e.g. `day=10, 11, 12` → `streak = 3`).
- **Multi-Day Gap (`day > last_day + 1`):** Resets the current streak back to `1` (e.g. `day=10, 11` then `day=15` → `streak = 1`).

The 4 Normative Requirements (`REQ-0001..REQ-0004`)

- **`REQ-0001` (Habit Registration):** `add_habit(name)` trims `name`, initializes streak to `0`, and raises `ValueError` on blank names.
- **`REQ-0002` (Idempotent Check-In):** First check-in sets streak to `1`; duplicate same-day check-in is a no-op; past days (`day < last_day`) raise `ValueError`.
- **`REQ-0003` (Consecutive Increment & Gap Reset):** `last_day + 1` increments streak; `> last_day + 1` resets streak to `1`.
- **`REQ-0004` (Unknown Habit Lookup):** Raises `KeyError` on unknown habit.

Goal-Driven Execution Setup (*Apply Spec*)

- In Lab 03, the spec (`specs/spec.md`) and design (`specs/design.md`) are already approved. Your job is to translate `REQ-0001..REQ-0004` into a sequenced TDD task plan and 1-to-1 traceable unit tests.

Lab 03 Step 2: Decomposing Requirements into Sequenced TDD Tasks

Structuring `expected_output/tasks.md` so every task pairs a `test_reqXXXX_*` test with minimal code

Sequenced TDD Task Plan (`expected_output/tasks.md`)

```
## Batch 1: Habit Registration & Same-Day Idempotency (REQ-0001, REQ-0002)
- [x] 1.1 Write test_req0001_add_habit_initializes_zero_streak_and_validates (Red)
        then implement add_habit() and blank-name check (Green).
- [x] 1.2 Write test_req0002_same_day_checkin_is_idempotent (Red)
        then implement first check-in and same-day guard (Green).

## Batch 2: Streak Arithmetic & Error Handling (REQ-0003, REQ-0004)
- [x] 2.1 Write test_req0003_consecutive_days_increment_and_gap_resets_streak (Red)
        then implement consecutive +1 and gap reset to 1 (Green).
- [x] 2.2 Write test_req0004_unknown_habit_raises_key_error (Red)

        then raise KeyError in record_checkin() and get_streak() (Green).
```

Why 1-to-1 Requirement-to-Test Naming Matters

- Naming every test `test_reqXXXX_<behavior>` (and putting `Verifies REQ-XXXX` in the docstring) creates an **auditable bi-directional link** between `spec.md` and your test runner.
- When `REQ-0003` changes in a future OpenSpec delta, the engineer (or agent) knows immediately which unit test must be updated first in the **Red** phase!

The Strict Red → Green → Refactor Rule (`/conductor:implement`)

- Never allow an agent to write `habit_tracker.py` first and “backfill” tests afterward. Write the failing test first, run `CI=true python3 -m unittest` to witness the failure, then write the code.

Lab 03 Step 3: Red-Green-Refactor Batch 1 (Registration & Idempotency)

Implementing `REQ-0001` and `REQ-0002` in `habit_tracker.py`

1. The Red Phase: Traceable Tests (`test_habit_tracker.py`)

```
def test_req0001_add_habit_initializes_zero_streak_and_validates(self):
  """Verifies REQ-0001: Habit creation trims name, starts at 0."""
  tracker = HabitTracker()
  tracker.add_habit(" Read 20 pages ")
  self.assertEqual(tracker.get_streak("Read 20 pages"), 0)
  with self.assertRaises(ValueError):
    tracker.add_habit(" ")

def test_req0002_same_day_checkin_is_idempotent(self):
  """Verifies REQ-0002: Duplicate same-day check-in keeps streak."""
  tracker = HabitTracker()
  tracker.add_habit("Exercise")
  self.assertEqual(tracker.record_checkin("Exercise", day=10), 1)

  self.assertEqual(tracker.record_checkin("Exercise", day=10), 1)
```

2. The Green Phase: Minimal Code (`habit_tracker.py`)

```
def add_habit(self, name: str) -> str:
  cleaned = name.strip() if isinstance(name, str) else ""
  if not cleaned:
    raise ValueError("Habit name must not be empty")
  if cleaned not in self._streaks:
    self._streaks[cleaned] = 0
    self._last_day[cleaned] = None
  return cleaned

# Inside record_checkin(self, name: str, day: int) -> int:
last = self._last_day[key]
if last is None:
  self._streaks[key] = 1
elif day == last:

  return self._streaks[key] # Idempotent no-op!
```

Why `REQ-0002` Explicitly Tests Same-Day Idempotency

- A classic bug in habit-tracking apps is incrementing the streak every time a user taps “Done” on the same day. Because `REQ-0002` specifies same-day idempotency, `test_req0002` locks that behavior down immediately.

Lab 03 Step 4: Red-Green-Refactor Batch 2 (Streaks & Missing Keys)

Implementing `REQ-0003` and `REQ-0004` with zero external dependencies

Consecutive Increment & Gap Reset (`habit_tracker.py`)

```
def record_checkin(self, name: str, day: int) -> int:
  key = name.strip() if isinstance(name, str) else ""
  if key not in self._streaks:
    raise KeyError(f"Unknown habit: {name}")
  if day < 0:
    raise ValueError("day must be non-negative")
  last = self._last_day[key]
  if last is None:
    self._streaks[key] = 1
  elif day < last:
    raise ValueError("Cannot check in before latest recorded day")
  elif day == last:
    return self._streaks[key]
  elif day == last + 1:
    self._streaks[key] += 1
  else:
    self._streaks[key] = 1
  self._last_day[key] = day

  return self._streaks[key]
```

Traceable Unit Tests (`REQ-0003` & `REQ-0004`)

- **`test_req0003_consecutive_days_increment_and_gap_resets_streak`:** Checks in on days `1, 2, 3` (asserting streak `1 -> 2 -> 3`), skips day `4`, and checks in on day `5` (asserting streak resets to `1`).
- **`test_req0004_unknown_habit_raises_key_error`:** Asserts both `record_checkin("Unknown", day=1)` and `get_streak("Unknown")` raise `KeyError`.

Deterministic Integer Ordinal Days

- By representing days as integer ordinal numbers (`day=1, 2, 3, ...`) instead of calling `datetime.now()`, all 4 unit tests run deterministically in **under 1 millisecond**!

Lab 03 Step 5: Git Notes Audit Trail & Running `./self_diagnose.sh`

Verifying 100% requirement-to-test traceability across `REQ-0001..REQ-0004`

Conductor `git notes` & Checkpoint Recording

```
$ git commit -m "feat(habits): implement streak increment and gap reset"
$ git notes add -m "Track: habit_tracker_20261008
Tasks: 2.1, 2.2 (REQ-0003, REQ-0004)
Tests: 4/4 passing (CI=true python3 -m unittest)" HEAD

# Phase Checkpoint in conductor/tracks/<id>/plan.md:

## Phase 2: Streak Arithmetic & Error Handling [checkpoint: 8f9e0a1]
```

- Attaching `git notes` keeps commit titles clean while storing structured task telemetry directly in Git object storage.

Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_03_from_specs_to_tdd_code/self_diagnose.sh
=== Lab 03 Self-Diagnosis: From Specs to TDD Code (HabitTracker) ===
test_req0001_add_habit_initializes_zero_streak_and_validates ... ok
test_req0002_same_day_checkin_is_idempotent ... ok
test_req0003_consecutive_days_increment_and_gap_resets_streak ... ok
test_req0004_unknown_habit_raises_key_error ... ok
----------------------------------------------------
Ran 4 tests in 0.001s -- OK

[PASS] All 4 HabitTracker requirements (REQ-0001..REQ-0004) verified!
```

Lab 03 Key Takeaway

- When every requirement ID (`REQ-0001..REQ-0004`) maps 1-to-1 to a Red-Green-Refactor unit test, verification becomes a deterministic < 1ms check rather than a subjective human eyeball review.

Lab 04 Step 1: Detecting Spec-to-Code Drift (REQ-0003 Off-by-One Bug)

Auditing `labs/lab_04_drift_detection_and_5step_refactoring/service/daily_planner.py`

What specs/spec.md Requires (REQ-0003)

```
### Requirement: [REQ-0003] Exact Budget Boundary & Over-Budget Rejection
The DailyPlanner.schedule(title, minutes) method SHALL admit tasks
when `used_minutes + minutes <= max_minutes` (including an exact
100% budget fit where `used_minutes + minutes == max_minutes`)
and MUST return `False` only when `used_minutes + minutes > max_minutes`.

#### Scenario: Task filling exact remaining minutes is accepted
- **GIVEN** a `DailyPlanner(max_minutes=60)` with `40` minutes used
- **WHEN** `schedule("Wrap-up review", 20)` is called (`40 + 20 == 60`)

- **THEN** it SHALL return `True` and `remaining_minutes()` SHALL be `0`.
```

The Bug in service/daily_planner.py

```
def schedule(self, title: str, minutes: int) -> bool:
  cleaned = title.strip() if isinstance(title, str) else ""
  if not cleaned or minutes <= 0:
    raise ValueError("Invalid task title or duration")
  # DRIFT BUG: uses >= instead of >, rejecting exact 100% budget fit!
  if self._used_minutes + minutes >= self.max_minutes:
    return False
  self._blocks.append((cleaned, minutes))
  self._used_minutes += minutes

  return True
```

- Because the developer wrote `>=` instead of `>`, a user with `20` minutes left in a `60`-minute budget cannot schedule a `20`-minute block!

Why Existing Happy-Path Tests Missed This Drift

- Happy-path tests only scheduled `30 + 20 = 50 < 60` minutes and `30 + 40 = 70 > 60` minutes—never testing the exact boundary `40 + 20 == 60` specified in `REQ-0003`.

Lab 04 Step 2: The 5-Step Brownfield Scorecard Refactoring Loop

Triaging `labs/lab_04_drift_detection_and_5step_refactoring/scorecard.md` (Steps 1 & 2)

Engineering Audit Scorecard (`scorecard.md`)

```
| ID | Severity | Finding Description | Decision |
|----|-------|-----------------------------|---------------|
| F-01 | CRITICAL | REQ-0003 off-by-one `>=` check rejects | ACCEPTED_FOR_REFACTOR |
| | | tasks that fill exact remaining budget. | |
| F-02 | LOW | Add Google Calendar OAuth two-way sync. | DEFERRED (Q4) |
| F-03 | LOW | Add Pomodoro countdown sound notifications.| DEFERRED |

| F-04 | NIT | Rename `schedule()` to `book_slot()`. | REJECTED |
```

The 5-Step Brownfield Refactoring Loop

Step 1 (Audit): Run spec-vs-code audit to populate `scorecard.md`.

Step 2 (Human Go/No-Go Triage): Lead engineer marks each finding `ACCEPTED_FOR_REFACTOR`, `DEFERRED`, or `REJECTED`.

Step 3 (Surgical Track Spec): Create Conductor track for `ACCEPTED` item(s) only; lock `DEFERRED`/`REJECTED` in `Out of Scope`.

Step 4 (Red-Green-Refactor TDD): Write regression test, apply minimal fix.

Step 5 (Review & Sync): Run `/conductor:review` & `/opsx:verify`.

Why Step 2 (Scorecard Triage) Stops "Mega-Prompt" Disasters

- If you feed all 4 findings to an agent at once, it will mix a 1-character boundary fix (`F-01`) with Google Calendar OAuth (`F-02`) and breaking API renames (`F-04`). The scorecard enforces **one surgical track per critical finding**.

Lab 04 Step 3: Enforcing Karpathy's "Surgical Changes" Rule

Locking `F-02`, `F-03`, and `F-04` inside Section 6 (`Out of Scope`) of `expected_output/spec.md`

Surgical Conductor Track `spec.md` (`expected_output/spec.md`)

```
## 1. Overview
Remediate Scorecard Finding F-01 (REQ-0003 off-by-one boundary bug)
in DailyPlanner.schedule().

## 3. Functional Requirements
- REQ-0003: Admit task when `used_minutes + minutes <= max_minutes`
  and reject (`False`) only when `used_minutes + minutes > max_minutes`.

## 6. Out of Scope
- F-02 (DEFERRED): Google Calendar OAuth two-way synchronization.
- F-03 (DEFERRED): Pomodoro timer sound notifications.

- F-04 (REJECTED): Renaming `schedule()` to `book_slot()`.
```

How `Out of Scope` Acts as a Hard Guardrail

- During `/conductor:implement`, the agent reads `spec.md`. Because `F-02`, `F-03`, and `F-04` are explicitly listed in `## 6. Out of Scope`, the agent is forbidden from renaming `schedule()` or adding OAuth/Pomodoro code.
- During `/conductor:review`, if the git diff touches anything in `Out of Scope`, the automated review flags a **Plan Compliance Violation**!

Result: A 1-Line Surgical Diff Instead of a 500-Line Rewrite

- Human review time drops from 30 minutes to **10 seconds**: the diff only changes `>=` to `>` in `schedule()` and adds `test_req0003_exact_budget_fit_and_over_budget_rejection`.

Lab 04 Step 4: Writing the Regression Test & Surgical Code Fix

Executing Step 4 (`Red` → `Green`) in `daily_planner_fixed.py` and `test_daily_planner.py`

1. The Red Phase: Failing `REQ-0003` Regression Test

```
def test_req0003_exact_budget_fit_and_over_budget_rejection(self):
  """Verifies REQ-0003: Exact 100% budget fit returns True; > returns False."""
  planner = DailyPlanner(max_minutes=60)
  self.assertTrue(planner.schedule("Deep work", 40))
  self.assertEqual(planner.remaining_minutes(), 20)

  # Exact boundary: 40 + 20 == 60 -> MUST return True!
  self.assertTrue(planner.schedule("Code review", 20))
  self.assertEqual(planner.remaining_minutes(), 0)

  # Over budget: 60 + 5 > 60 -> MUST return False!

  self.assertFalse(planner.schedule("Extra call", 5))
```

2. The Green Phase: Surgical Fix in `schedule()`

```
def schedule(self, title: str, minutes: int) -> bool:
  cleaned = title.strip() if isinstance(title, str) else ""
  if not cleaned:
    raise ValueError("Task title must not be empty")
  if minutes <= 0:
    raise ValueError("minutes must be positive")
  # FIXED REQ-0003: reject only when strictly exceeding max_minutes
  if self._used_minutes + minutes > self.max_minutes:
    return False
  self._blocks.append((cleaned, minutes))
  self._used_minutes += minutes

  return True
```

Crystal-Clear Boundary Verification

- With `> self.max_minutes`, scheduling `40 + 20` minutes in a `60`-minute day returns `True` (`remaining_minutes() == 0`), while any additional task returns `False` without mutating state.

Lab 04 Step 5: Differential Bug/Fix Verification in `./self_diagnose.sh`

Proving the regression test fails on the bugged service and passes on the fixed service

How `lab_04/self_diagnose.sh` Proves the Fix

1. **Negative Control (Bugged Service):** Runs `test_req0003` against `service/daily_planner.py` and verifies that it **fails** (proving the test genuinely catches the off-by-one bug).
2. **Positive Control (Fixed Service):** Runs the full 4-test suite (`REQ-0001..REQ-0004`) against `expected_output/daily_planner_fixed.py` and verifies **4/4 pass**.
3. **Scorecard & Scope Audit:** Confirms `expected_output/spec.md` contains all 7 sections and lists `F-02`, `F-03`, and `F-04` under `Out of Scope`.

Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_04_drift_detection_and_5step_refactoring/self_diagnose.sh
=== Lab 04 Self-Diagnosis: Spec Drift & 5-Step Refactoring ===
test_req0001_initializes_budget_and_validates_inputs ... ok
test_req0002_schedules_tasks_and_tracks_remaining_minutes ... ok
test_req0003_exact_budget_fit_and_over_budget_rejection ... ok
test_req0004_lists_scheduled_tasks_in_order ... ok
----------------------------------------------------------
Ran 4 tests in 0.001s -- OK
[PASS] Drift bug confirmed in service/daily_planner.py, fixed in

       expected_output/, and scoped via 5-Step Scorecard Track Spec!
```

Lab 04 Key Takeaway

- A regression test is only proven when you witness it **fail** on the buggy code (**Red**) and **pass** on the surgical fix (**Green**).

Lab 05 Step 1: Auditing an Incoming “Vibe-Coded” Pull Request

Inspecting `labs/lab_05_sdd_code_review_and_stacked_prs/simulated_review/subpar_change.patch`

Baseline System Truth (`base/openspec/specs/notes-board/spec.md`)

- **`REQ-0001` (Note Creation):** `add_note(text)` trims text, assigns sequential `id`, sets `pinned=False`, and rejects blank strings.
- **`REQ-0002` (Chronological Listing Invariant):** `list_notes()` MUST return all stored notes in **chronological creation order** (`id = 1, 2, ...`).
- **`REQ-0003` (Note Deletion):** `delete_note(note_id)` removes the note and returns `True` or `False`.

The Incoming Patch (`simulated_review/subpar_change.patch`)

```
@@ -22,5 +22,12 @@ class NotesBoard:
def list_notes(self) -> list[Note]:
- return list(self._notes.values())
+ return list(reversed(list(self._notes.values()))) # "Newest first!"

+ def pin_note(self, note_id: int) -> None:
+ if note_id in self._notes:
+ self._notes[note_id].pinned = True

+ def get_pinned_notes(self) -> list[Note]:

+ return [n for n in self._notes.values() if n.pinned]
```

Why Traditional “Code-Only” Review Misses This

- A reviewer looking only at Python syntax thinks `reversed(...)` and `pin_note()` look fine! Only by reviewing against `openspec/specs/notes-board/spec.md` do you catch that `REQ-0002` mandates strict **creation order**.

Lab 05 Step 2: Catching All 3 SDD Violations via `/conductor:review`

How Spec-Driven Code Review blocks behavioral drift, un-specced APIs, and silent error swallowing

Violation 1: Spec Contradiction

Breaks `REQ-0002` Order

- Wrapping `list_notes()` in `reversed(...)` silently breaks `REQ-0002` (which mandates chronological creation order).
- Any caller displaying notes in creation sequence suffers an immediate regression.

Violation 2: Un-Specced API

Zero Spec Delta for `pin_note`

- Adds public methods `pin_note(note_id)` and `get_pinned_notes()` with no `REQ-0004` in `openspec/specs/notes-board/spec.md`.
- Leaves pinned note ordering and error semantics undocumented.

Violation 3: Silent Error & 0 Tests

Missing `KeyError` & No Tests

- `pin_note(999)` silently does nothing on an unknown ID instead of raising `KeyError` (and mutates a frozen dataclass).
- Includes zero `test_req0004_*` unit tests.

The SDD Code Review Golden Rule

- **Review the Spec Diff First:** Every PR that adds or changes observable behavior MUST include an OpenSpec delta (`REQ-XXXX`) and a matching `test_reqXXXX_*` test. If code and spec disagree, block the PR.

Lab 05 Step 3: Synchronizing the OpenSpec Contract (REQ-0004)

Adding REQ-0004 to expected_output/openspec/specs/notes-board/spec.md while preserving REQ-0002

Synchronized Spec Delta (REQ-0004: Pin Note & List Pinned Notes)

```
### Requirement: [REQ-0004] Pin Note and List Pinned Notes
The NotesBoard SHALL support `pin_note(note_id)` to mark an existing
note with `pinned=True` (raising `KeyError` if `note_id` does not
exist) and `get_pinned_notes()` to return all pinned notes in
chronological creation order without altering `list_notes()`.

#### Scenario: Pinning a note and querying pinned notes
- **GIVEN** a `NotesBoard` with notes `1` (`"First"`) and `2` (`"Second"`)
- **WHEN** `pin_note(2)` is called
- **THEN** `get_pinned_notes()` SHALL return `[Note(id=2, ...)]`,
  `list_notes()` SHALL still return `[Note(id=1), Note(id=2)]`,

  and `pin_note(999)` SHALL raise `KeyError`.
```

How Socratic Grilling Clarified REQ-0004

- When running /opsx:propose, OpenSpec's Socratic Grill asked whether list_notes() should stay in creation order (REQ-0002) while get_pinned_notes() filters pinned notes, and what pin_note(999) should do on a missing ID.
- Answering those two questions produced the exact, unambiguous REQ-0004 scenario on the left!

Preserving REQ-0001..REQ-0003 Intact

- In expected_output/openspec/specs/notes-board/spec.md, REQ-0001 through REQ-0003 remain intact (strict creation order), and REQ-0004 is cleanly appended.

Lab 05 Step 4: Remediating `notes_board.py` & Verifying `test_notes_board.py`

Restoring `REQ-0002` creation order and implementing `REQ-0004 pin_note()` with `KeyError`

Compliant Implementation (`expected_output/notes_board.py`)

```
def list_notes(self) -> list[Note]:
  return list(self._notes.values()) # <- Preserves REQ-0002 order!

def pin_note(self, note_id: int) -> Note:
  if note_id not in self._notes:
    raise KeyError(f"Unknown note_id: {note_id}")
  current = self._notes[note_id]
  updated = Note(id=current.id, text=current.text, pinned=True)
  self._notes[note_id] = updated
  return updated

def get_pinned_notes(self) -> list[Note]:

  return [n for n in self._notes.values() if n.pinned]
```

Traceable Test Suite (`expected_output/test_notes_board.py`)

```
test_req0001_add_note_trims_and_rejects_blank ... ok
test_req0002_list_notes_preserves_chronological_creation_order ... ok
test_req0003_delete_note_removes_and_returns_bool ... ok

test_req0004_pin_note_marks_pinned_and_raises_on_missing_id ... ok
```

- `test_req0002` asserts `[n.id for n in board.list_notes()] == [1, 2, 3]` even after note `3` is pinned (blocking any future `reversed()` regression).

Immutable `Note` Dataclass Replacement

- Because `Note` is a `@dataclass(frozen=True)`, `pin_note()` constructs a new `Note(..., pinned=True)` instance instead of mutating state in place.

Lab 05 Step 5: Stacked Multi-PR Decomposition & Archive PR N+1

Structuring `expected_output/tasks.md` for clean Git / Jujutsu (`jj`) stacked pull requests

Stacked PR Task Breakdown (`expected_output/tasks.md`)

```
## Batch 1: Spec Delta & Creation-Order Guard (PR 1)
- [x] 1.1 Add REQ-0004 to openspec/specs/notes-board/spec.md & design.md
- [x] 1.2 Restore chronological list_notes() order (REQ-0002)

## Batch 2: Note Pinning Implementation & Tests (PR 2)
- [x] 2.1 Implement pin_note(note_id) with KeyError & get_pinned_notes()
- [x] 2.2 Add test_req0004_pin_note_marks_pinned_and_raises_on_missing_id

## Batch 3: Dedicated OpenSpec Archive (PR N+1 -- Docs Only)

- [x] 3.1 Run /opsx:verify & /opsx:archive after PR 1 & PR 2 merge
```

The 4 Stacked Multi-PR Rules

1. **One Change Folder = One Feature:** Keep a single `openspec/changes/<id>/` folder across the stack.
2. **Batch `tasks.md` by PR:** Group checkboxes under `Batch 1 (PR 1)`, `Batch 2 (PR 2)`.
3. **Truthful Same-PR Checkmarks:** Only mark `[x]` for tasks whose code + tests are in that PR.
4. **Dedicated Archive PR `N+1`:** Never run `/opsx:archive` mid-stack (moving files to `archive/` breaks downstream rebase!). Run `/opsx:archive` in a final docs-only PR `N+1`.

Lab 05 Verification Command

- Run `./labs/lab_05_sdd_code_review_and_stacked_prs/self_diagnose.sh` to verify the `REQ-0004` contract, Stacked PR `tasks.md` batches, and all 4 `NotesBoard` unit tests.

Lab 06 Step 1: The 8-Section `SPEC.md` (Sections 1–4)

Inspecting `labs/lab_06_capstone_ssot_and_rebuild_test/SPEC.md` for `TaskFlowBoard`

Sections 1 & 2: Domain Intent & Architecture

- **Section 1 (System Overview & Domain Intent):** `TaskFlowBoard` is `TaskFlow`'s unified 3-column Kanban board (`"todo"`, `"doing"`, `"done"`) with configurable Work-In-Progress (`doing_wip_limit`) enforcement.
- **Section 2 (Architecture & Component Topology):** Pure Python 3.10+ domain service backed by an insertion-ordered `dict[int, Card]` with zero external dependencies.

Sections 3 & 4: Data Schemas & API Contracts

- **Section 3 (Data Models):** Immutable `Card(id: int, title: str, column: str)` dataclass and `VALID_COLUMNS = ("todo", "doing", "done")`.
- **Section 4 (Interface Contracts):** Exact signatures and error semantics for `create_card(title)`, `move_card(card_id, target_column)`, `get_column(column)`, and `get_wip_summary()` (raising `KeyError` on unknown `card_id` and `ValueError` on WIP overflow).

Essential vs. Incidental Complexity in `SPEC.md`

- Sections 1–4 specify every **observable type, field name, column string (`"todo"`, `"doing"`, `"done"`), and exception type** so an independent test suite can bind to the regenerated code with zero glue adjustments.

Lab 06 Step 2: The 8-Section `SPEC.md` (Sections 5–8)

WIP-limit workflows, parameterized config, and the 3-Phase Clean-Room Rebuild Recipe

Sections 5, 6 & 7: Workflows, Layout & Config

- **Section 5 (Behavioral Workflows & `REQ-0001..0004`):**
  - `REQ-0001`: `create_card(title)` trims title, starts in `"todo"`, validates `doing_wip_limit >= 1`.
  - `REQ-0002`: `move_card(id, col)` transitions cards across `"todo"`, `"doing"`, `"done"`.
  - `REQ-0003`: Moving a new card into `"doing"` when `doing` is at `doing_wip_limit` MUST raise `ValueError` (moving an already-`"doing"` card to `"doing"` is a no-op).
  - `REQ-0004`: `get_column(col)` & `get_wip_summary()` return snapshot counts.
- **Sections 6 & 7:** Module layout (`taskflow_app.py`) & env parameterization.

Section 8: The 3-Phase Clean-Room Rebuild Recipe

Phase A (Data Models & Validation): Generate `Card`, `VALID_COLUMNS`, and constructor/title validation (`REQ-0001`).

Phase B (Column Transitions & WIP Limit): Implement `move_card()` with `doing_wip_limit` guard, `get_column()`, and `get_wip_summary()` (`REQ-0002..0004`).

Phase C (Adversarial Verification): Execute read-only `test_rebuild_contract.py` in an isolated `/tmp/` clean room.

The Level-3 SSOT Litmus Test (*Clean Spec*)

- If an engineer or AI agent can read `SPEC.md` in an empty directory and produce `taskflow_app.py` that passes all 4 adversarial contract tests on the first run, `SPEC.md` is a true **Single Source of Truth**.

Lab 06 Step 3: Clean-Room Rebuild & `chmod -w` Adversarial Isolation

How `labs/lab_06_capstone_ssot_and_rebuild_test/self_diagnose.sh` enforces Karpathy's Immutable Verifier

Clean-Room Rebuild Harness (`lab_06/self_diagnose.sh`)

```
# 1. Create isolated clean-room workspace in /tmp
CLEAN_ROOM="$(mktemp -d /tmp/sdd_rebuild_XXXXXX)"
trap 'chmod -R u+w "$CLEAN_ROOM" 2>/dev/null; rm -rf "$CLEAN_ROOM"' EXIT

# 2. Copy SPEC.md, regenerated implementation, and adversarial tests
cp "$SCRIPT_DIR/SPEC.md" "$CLEAN_ROOM/SPEC.md"
cp "$TARGET_PATH/taskflow_app.py" "$CLEAN_ROOM/taskflow_app.py"
mkdir -p "$CLEAN_ROOM/tests"
cp "$SCRIPT_DIR/adversarial_tests/test_rebuild_contract.py"
   "$CLEAN_ROOM/tests/test_rebuild_contract.py"

# 3. Lock adversarial test directory READ-ONLY (chmod -w)

chmod -R a-w "$CLEAN_ROOM/tests"
```

Why `chmod -R a-w` Is Critical for Agentic Verification

- Just like Karpathy's `autoresearch` locks `prepare.py` (`val_bpb` evaluator) out of the agent's write scope, locking `tests/` with `chmod -R a-w` physically prevents an implementation agent from editing or deleting test assertions.
- We also separate roles: **Agent A (Test Scaffolder)** writes `test_rebuild_contract.py` from `SPEC.md`, while **Agent B (Implementer)** writes `taskflow_app.py` without permission to modify `tests/`.

Zero Shared State with the Original Repo

- Running `python3 -m unittest` inside `$CLEAN_ROOM` guarantees `taskflow_app.py` doesn't rely on hidden files or undeclared imports in the parent repository.

Lab 06 Step 4: Inspecting `taskflow_app.py` & `test_rebuild_contract.py`

Verifying WIP-limit enforcement (REQ-0003) and column summary snapshots (REQ-0004)

WIP-Guarded Column Move (`taskflow_app.py`)

```
def move_card(self, card_id: int, target_column: str) -> Card:
  if card_id not in self._cards:
    raise KeyError(f"Unknown card_id: {card_id}")
  col = target_column.strip().lower() if isinstance(target_column, str) else ""
  if col not in VALID_COLUMNS:
    raise ValueError(f"Invalid column: {target_column}")
  current = self._cards[card_id]
  if col == "doing" and current.column != "doing":
    doing_count = sum(1 for c in self._cards.values() if c.column == "doing")
    if doing_count >= self.doing_wip_limit:
      raise ValueError("doing_wip_limit reached")
  updated = Card(id=current.id, title=current.title, column=col)
  self._cards[card_id] = updated

  return updated
```

Live Clean-Room Verification (`./self_diagnose.sh`)

```
$ ./labs/lab_06_capstone_ssot_and_rebuild_test/self_diagnose.sh
=== Lab 06 Self-Diagnosis: 8-Section SPEC.md & Rebuild Test ===
test_req0001_create_card_and_validate_wip_limit ... ok
test_req0002_move_card_across_columns_and_reject_unknown ... ok
test_req0003_enforces_doing_wip_limit_until_slot_freed ... ok
test_req0004_get_column_and_wip_summary_counts ... ok
----------------------------------------------------
Ran 4 tests in 0.001s -- OK
[PASS] Clean-Room Rebuild Test passed all read-only adversarial

       contract tests (REQ-0001..REQ-0004)!
```

Unblocking WIP Capacity (REQ-0003)

- `test_req0003` fills `"doing"` to `doing_wip_limit=2`, verifies moving a 3rd card into `"doing"` raises `ValueError`, moves card 1 to `"done"`, and verifies the 3rd card can now enter `"doing"`.

Master Diagnostic Checklist & Running `./self_diagnose_all.sh`

Your complete toolkit for teaching and practicing Spec-Driven Development on `TaskFlow`

Top 4 Practitioner Traps to Avoid

1. **Partial `MODIFIED` Block in OpenSpec:** `MODIFIED` replaces the *entire* requirement block on archive. Always copy the full requirement + all existing scenarios before editing!
2. **3-Hashtag `### Scenario:` Header:** OpenSpec's parser strictly requires **4 hashtags** (`#### Scenario:`); 3 hashtags silently drop scenarios.
3. **`RENAMED` + `MODIFIED` Header Collision:** `MODIFIED` must reference the **new (`TO`)** requirement name.
4. **Archiving Mid-Stack:** Only run `/opsx:archive` in final PR `N+1`.

Run the Full 6-Lab Self-Diagnosis Harness

```
$ git clone https://github.com/adkaspar-google/sdd-course.git
$ cd sdd-course && ./self_diagnose_all.sh

[PASS] Lab 01: Greenfield TaskFlow TodoList (4/4 tests)
[PASS] Lab 02: Brownfield TaskFlow TaskOrganizer (4/4 tests)
[PASS] Lab 03: TaskFlow HabitTracker TDD (4/4 tests)
[PASS] Lab 04: TaskFlow DailyPlanner Drift & Refactoring (4/4 tests)
[PASS] Lab 05: TaskFlow NotesBoard Review & Stacked PRs (4/4 tests)
[PASS] Lab 06: TaskFlowBoard SPEC.md Clean-Room Rebuild (4/4 tests)
============================================================

[ALL PASSED] 6/6 Hands-On Labs & 24/24 Traceable Tests!
```

Congratulations on Completing `SDD-Crash-Course`!

- **Course Materials in This Repo:** 6 Executable `TaskFlow` Labs (`labs/`), 17-Page Practical Coursebook PDF (`Coursebook_SDD_Conductor_OpenSpec.pdf`), Dual-Engine Playbook (`playbooks/`), `@writing-arch-specs` Skill (`.agents/skills/`), and 7 Step-by-Step Video Modules (`slides/`).